

Advanced Simulation of Power System Stability and Cascading Failure Dynamics in Resilient Smart Grids

Written Assignment

Student Name: Salina Khadka

Student ID: [Your Student ID]

Module: Advanced Simulation Techniques (AST)

Module Code: [Module Code]

Degree: BSc Computer Science and Digitisation

Issued by: Dr. Waseem Alhasan

Institution: Berlin School of Business and Innovation (BSBI) / UCA

Submission Date: 17 June 2026

Table of Contents

Task 1: Critical Infrastructure Analysis.....	3
1.1 Key Components, Stakeholders and Interdependencies.....	3
1.2 Vulnerabilities and Failure Modes.....	4
1.3 Modelling Approach and Justification.....	5
Task 2: Simulation Design and Implementation.....	6
2.1 Conceptual Model and System Diagram.....	6
2.2 Implementation: Key Decisions and Parameter Settings.....	7
2.3 User Interface, Visualisation and Export.....	8
2.4 Scenario Execution, Results and Performance Metrics.....	10
Task 3: Tool and Technique Comparison.....	11
3.1 Comparison of Two Simulation Tools/Techniques.....	11
3.2 Alternative Approaches and Reflection.....	12
Task 4: Business Value and Future Directions.....	13
4.1 Competitive Advantage and Return on Investment.....	13
4.2 Emerging Trends and Ethical Considerations.....	14
References.....	15
Appendix A: Task 2 Source Code.....	17
config.py.....	17
grid_model.py.....	18
metrics.py.....	22
scenarios.py.....	23
visualization.py.....	25
make_system_diagram.py.....	28
run_simulation.py.....	29

Task 1: Critical Infrastructure Analysis

1.1 Key Components, Stakeholders and Interdependencies

The electrical power grid is the archetypal critical infrastructure: a tightly coupled, geographically dispersed network whose continuous operation underpins almost every other sector of modern society. A modern smart grid integrates four functional layers — generation, transmission, distribution and consumption — with an overarching layer of information and communication technology (ICT) that senses, coordinates and controls the physical flow of electricity in near real time (Amin and Wollenberg, 2005). The present analysis focuses on a regional distribution system comprising dispatchable coal generation, variable solar generation, a small transmission network, substations and the consumer zones they serve.

The key physical components are generators, transmission lines, substations (transformers and switchgear), protection relays, and the loads themselves. In the modelled system these are represented concretely as two coal units (300 MW and 200 MW), a 150 MW solar farm, five transmission lines of differing thermal ratings, and four substations feeding industrial, residential, commercial and mixed consumer zones. Layered above the physical assets is the cyber-control system — supervisory control and data acquisition (SCADA), state estimation and protection logic — which monitors line loadings and dispatches corrective action. This cyber-physical coupling is the defining feature of the smart grid and the source of much of its complexity (Amin and Wollenberg, 2005).

The stakeholders form an interdependent ecosystem. Generation companies and renewable operators inject power; the transmission and distribution system operators (TSO/DSO) are responsible for stability, balancing and restoration; regulators set reliability standards and market rules; and end-consumers — hospitals, water-treatment plants, telecommunications, transport and households — depend on uninterrupted supply. Repair crews and maintenance contractors constitute a critical but finite resource whose availability directly governs how quickly the system recovers from faults.

Interdependencies operate both within the grid and across infrastructures. Internally, generation must be continuously balanced against demand; the failure of a single transmission line forces power to redistribute onto its neighbours, which may in turn overload and trip — the mechanism of a cascading failure (Dobson et al., 2007). Externally, the grid exhibits strong cross-

sector interdependencies: telecommunications and SCADA require power, yet the grid's own control depends on those same communications, producing the bidirectional physical, cyber, geographic and logical couplings catalogued by Rinaldi, Peerenboom and Kelly (2001). Water, fuel, finance, healthcare and transport all rely on electricity, so a sustained outage propagates rapidly into a society-wide crisis.

It is precisely this combination of universal dependence, tight coupling and potential for cascading propagation that makes the power grid critical to society. Reliable electricity is a precondition for public health, economic activity and national security, and even short interruptions impose large economic and human costs (U.S.–Canada Power System Outage Task Force, 2004). Understanding how the grid behaves under stress — and how mitigation measures such as additional repair capacity change that behaviour — is therefore of direct societal value, and is the motivation for the simulation developed in this report.

1.2 Vulnerabilities and Failure Modes

Power systems are vulnerable to a spectrum of failure modes that can be broadly grouped into physical, operational, environmental and cyber categories. Physically, transmission lines and transformers have finite thermal ratings; sustained overload causes lines to sag, trip or fail, while ageing assets exhibit rising fault probabilities (Kundur et al., 2004). Operationally, the instantaneous balance between generation and demand must be maintained; the growing penetration of variable renewables introduces intermittency that complicates this balance (Amin and Wollenberg, 2005). Environmentally, extreme weather is an increasingly important driver of correlated, simultaneous failures (Panteli and Mancarella, 2015). Finally, the cyber layer introduces a new attack surface in which compromised control systems can be weaponised against the physical grid.

The most consequential failure mode is the cascading failure, in which an initial disturbance triggers a self-propagating sequence of overloads and trips. The 2003 Northeast blackout is the canonical example: a software bug in an alarm system, combined with inadequate tree trimming and poor situational awareness, allowed a local line fault in Ohio to cascade into a loss of supply to roughly 50 million people across the north-eastern United States and Canada (U.S.–Canada Power System Outage Task Force, 2004). The event demonstrated how tightly coupled networks redistribute load after each trip, repeatedly pushing neighbouring elements past their limits — exactly the dynamic this report models.

More recent events confirm the persistence of these vulnerabilities. The 2016 South Australian ‘black system’ event showed how extreme weather and the ride-through settings of wind generation could combine to collapse an entire regional grid (AEMO, 2017), highlighting the renewable-integration risk. The January 2021 separation of the Continental European synchronous area into two islands illustrated how a single substation fault can propagate across national borders before automatic defence mechanisms arrest it (ENTSO-E, 2021). Together these case studies show that vulnerability is not merely a function of any single component but emerges from the interaction of demand stress, asset condition, renewable variability and the finite speed of human and automated response.

From a modelling perspective, these historical examples motivate the specific mechanisms embedded in the simulation: a time-varying demand profile, intermittent solar output, capacity-proportional load redistribution after a trip, an overload-dependent failure probability, and a finite repair resource. By representing these explicitly, the model can reproduce the qualitative signature of cascading failure and be used to test which mitigation levers — additional crews, reduced loading, or improved asset condition — most effectively improve resilience (Dobson et al., 2007).

1.3 Modelling Approach and Justification

Selecting an appropriate modelling paradigm is decisive for both validity and tractability. The principal candidates are continuous simulation (differential equations), system dynamics, agent-based modelling (ABM), discrete-event simulation (DES) and hybrid combinations of these (Borshchev and Filippov, 2004). The choice should follow the dominant characteristics of the system under study.

The resilience questions addressed here are governed by discrete occurrences — a line overloads, a line fails, a repair completes, a neighbour trips — separated by intervals during which the system state changes only gradually. This is the defining profile of a discrete-event system, in which the state changes only at a countable set of event times (Banks et al., 2010). Discrete-event simulation is therefore selected as the primary approach. It represents failures and repairs as instantaneous events, models the repair crew naturally as a shared, capacity-limited resource for which competing repairs must queue, and is computationally efficient because the simulation clock advances from event to event rather than over fine fixed time-steps (Law, 2015).

A purely continuous or system-dynamics model, by contrast, excels at representing aggregate feedback and smoothly varying quantities but is awkward for discrete, queue-based resource contention and individual component trips (Sterman, 2000). A fully agent-based model could capture heterogeneous component behaviour but would add implementation overhead disproportionate to the questions asked here (Macal and North, 2010). The adopted design is therefore best described as discrete-event with a light hybrid flavour: the discrete event engine (SimPy) drives the simulation, while the continuous demand and solar profiles are sampled at an hourly step and injected as time-varying inputs. This hybrid framing retains the clarity and efficiency of DES while still capturing the continuous variability that drives the grid towards its failure thresholds.

Task 2: Simulation Design and Implementation

2.1 Conceptual Model and System Diagram

The conceptual model defines the boundaries, entities, attributes and relationships of the simulated grid before any code is written. The system boundary encloses the generation, transmission and substation layers of a single regional distribution grid together with the repair organisation; external markets, weather systems and the wider interconnection are treated as exogenous inputs (demand and solar profiles) rather than modelled explicitly. This scoping keeps the model focused on the cascading-failure and recovery dynamics that are the object of study.

Three classes of entity are represented. Generators have attributes of rated capacity, type (coal or solar) and dispatchability; the two coal units provide controllable baseload while the solar farm follows a daylight curve modulated by random cloud cover. Transmission lines have a thermal capacity, an instantaneous load and an in-service status; they are the elements that overload, fail and are repaired. Substations map generation onto four consumer zones, each drawing a fixed share of total demand. The shared repair crew is modelled as a capacity-limited resource for which failed lines must queue.

The relationships among these entities are shown in Figure 1. Power flows left to right from generators, through transmission lines, into substations and on to the consumer zones. When a line fails, demand is redistributed across the remaining in-service lines in proportion to their capacities; if this redistribution pushes a neighbour beyond the cascade threshold, a cascading

event is recorded. The control loop closes when the repair crew restores a failed line, returning it to service and triggering a further redistribution. These four layers and the feedback control loop constitute the complete conceptual model implemented in Task 2.2.

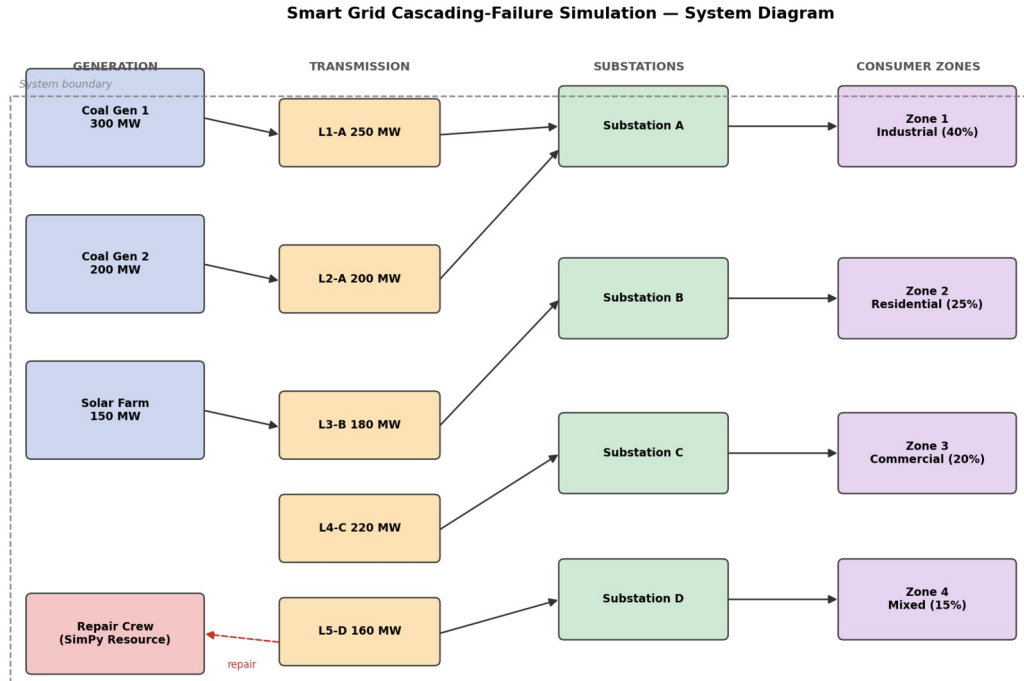


Figure 1. Conceptual system diagram: generation → transmission → substations → consumer zones, with the repair-crew control loop.

2.2 Implementation: Key Decisions and Parameter Settings

The simulation is implemented in Python using the SimPy 4 discrete-event framework (Team SimPy, 2023), chosen for its lightweight process-based API and transparent, auditable code. The implementation is deliberately decomposed into separate modules — configuration, model, metrics, scenarios and visualisation — so that a single source of truth for parameters is shared by the command-line runner and the interactive Colab interface. The complete listings appear in Appendix A.

The core of the model is the GridSimulation class, which owns its own random-number generator so that every run is both reproducible (via an explicit seed) and isolated from other runs during the scenario sweep. Five categories of concurrent SimPy process run on a shared environment: a generator process sets hourly output; a demand process applies a time-of-day load profile with stochastic jitter; one line-monitor process per transmission line checks for overload and stochastic failure each hour; a repair process is spawned on failure and competes

for the repair-crew resource; and a metrics collector samples the full system state every hour for plotting and export. Using one monitor process per line is the key design decision that lets cascading behaviour emerge naturally rather than being scripted.

Several modelling decisions warrant justification. Load is redistributed in proportion to the rated capacity of the remaining in-service lines, a simple but defensible surrogate for a full power-flow solution that preserves the essential cascade mechanism while keeping the model transparent. Failure is treated probabilistically: each line carries a base hourly failure probability that is multiplied by a configurable factor when the line is overloaded, capturing the empirically observed acceleration of failures under stress (Dobson et al., 2007). The repair crew is a SimPy Resource with a configurable capacity, so that when failures outnumber crews, repairs queue and outage durations lengthen — directly linking resourcing decisions to resilience.

The default parameter set defines the baseline scenario: a 72-hour horizon advanced in one-hour steps, a 5% base hourly failure probability, a cascade threshold at 120% of rated capacity, an overload failure multiplier of five, repair durations drawn uniformly between two and four hours, a single repair crew, and a fixed seed of 42. These values were chosen to place the baseline grid in a realistic, lightly stressed regime — occasional failures that are normally repaired before they cascade — so that the what-if scenarios in Task 2.4 can reveal how demand growth, ageing assets or extra crews shift the system towards or away from systemic failure. All parameters are exposed as configuration keys and as interactive sliders, making the model straightforward to recalibrate against empirical data.

2.3 User Interface, Visualisation and Export

To satisfy the requirement for interactive control and visualisation, a Google Colab notebook provides the user interface, built with the ipywidgets library. The notebook clones the model repository, then renders a control panel of sliders and input boxes that allow the user to adjust the simulation horizon, demand multiplier, base failure probability, number of repair crews, solar scaling, cascade threshold and random seed. Pressing the Run button re-executes the simulation with the chosen parameters and immediately redraws every output, giving the effective real-time control over parameters that the brief requires; an Export button writes and downloads the results.

Four complementary visualisations are produced (Figures 2–4 plus a failure-timeline plot). A stacked-area chart shows generator output against total demand, making renewable intermittency and supply–demand tightness visible at a glance. A load-percentage chart plots the average line loading against the 100% rating and the cascade threshold, with vertical markers at each failure time. A stem plot gives the discrete failure-event timeline. Finally, a colour-coded heatmap displays the load of every individual line over time, so that localised overloads and the spatial spread of a cascade can be read directly from the figure. Together these views address the differing needs of operational, tabular and spatial analysis.

Results can be exported for further analysis in three forms: an hourly time-series CSV (demand, per-generator output, average and per-line loading), a discrete event log CSV (every failure, cascade and restoration with its timestamp), and a scenario-comparison CSV. Exporting to CSV rather than a proprietary format ensures the outputs can be opened in any spreadsheet or statistical package, supporting onward analysis and reproducibility. The same figure-building functions are reused by the headless command-line runner, so the interactive and batch workflows produce identical artefacts.

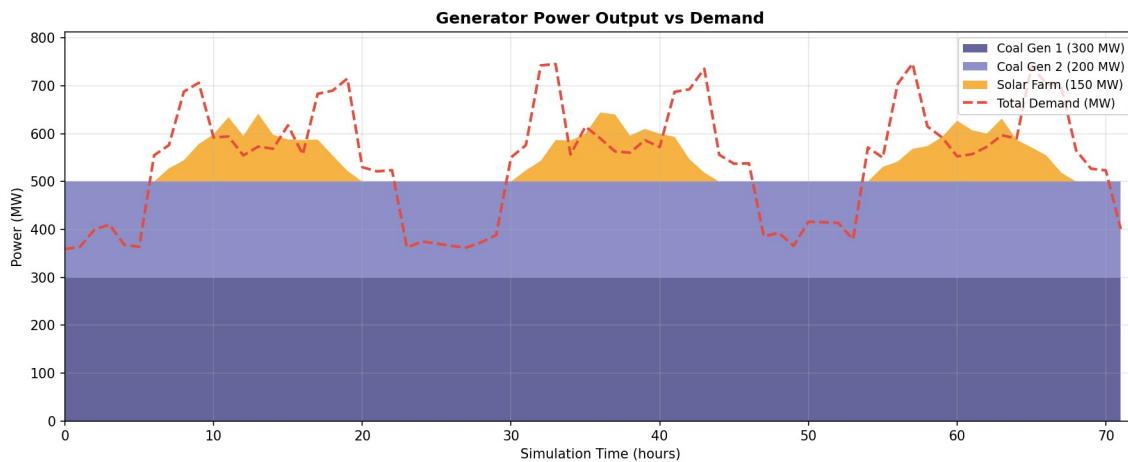


Figure 2. Colab UI output — stacked generator output versus total demand.

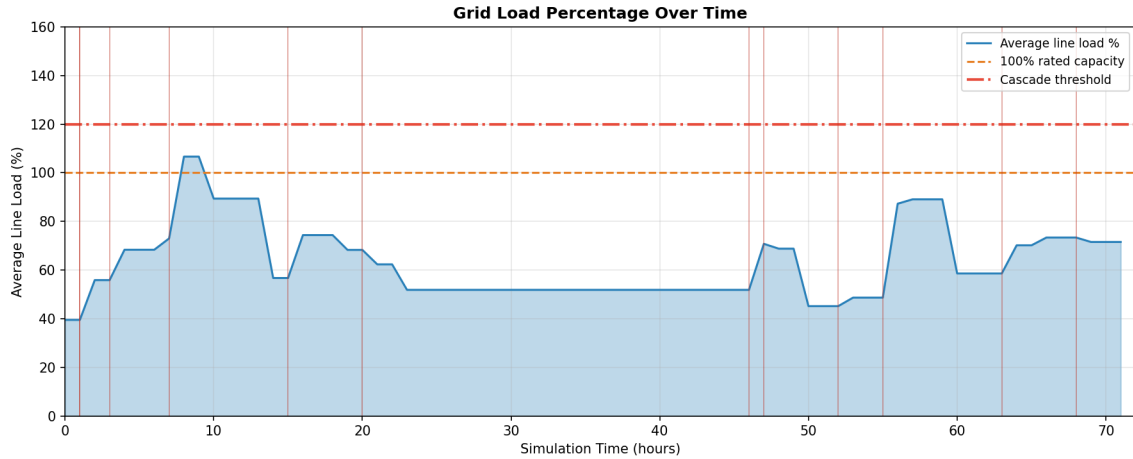


Figure 3. Colab UI output — average line load with overload and cascade thresholds.

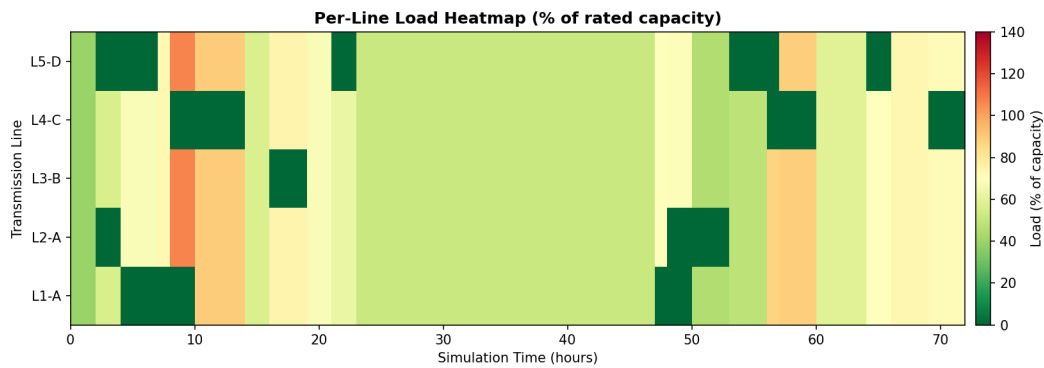


Figure 4. Colab UI output — per-line load heatmap (% of rated capacity).

2.4 Scenario Execution, Results and Performance Metrics

The model was executed under six scenarios designed to probe distinct stressors: a Baseline; High Demand (+25%); Aged Infrastructure (tripled failure probability); High Renewable (solar boosted 40%); Extra Repair Crews (three instead of one); and a combined Stress Test. All runs used a 72-hour horizon and a fixed seed of 42 to isolate the effect of each lever. Table 1 summarises the headline metrics.

Scenario	Failures	Cascades	MTTR (h)	Peak load %	Crew util.
Baseline	12	0	4.25	106.7	0.535
High Demand (+25%)	15	38	6.24	361.9	0.617
Aged Infrastructure	28	29	11.1	453.9	0.993

High Renewable	12	0	4.25	106.7	0.535
Extra Repair Crews	16	15	3.04	160.3	0.222
Stress Test	28	41	12.29	556.9	0.99

Table 1. Headline resilience metrics across the six what-if scenarios (seed = 42, 72-hour horizon).

The results display a clear and interpretable resilience signature. The baseline grid suffers twelve failures, all repaired, with no cascades and a peak average loading of 107%. Increasing demand by 25% is sufficient to push lines past their cascade threshold, producing 38 cascade events and a peak loading above 360%, while ageing the infrastructure raises failures to 28 and leaves five lines unrecovered at the end of the horizon. The Stress Test, combining both, is the most severe, with 41 cascades and crew utilisation near saturation — the model's depiction of a grid overwhelmed faster than it can be repaired. Crucially, adding repair crews cuts the mean time to repair (MTTR) from 4.25 to 3.04 hours and lowers crew utilisation, demonstrating the simulation's value as a decision-support tool for resourcing.

Performance and accuracy were also measured. Each 72-hour run completes in roughly 3–4 milliseconds on a standard machine, so the entire six-scenario sweep and a 30-seed accuracy study run in well under a second, confirming that discrete-event simulation is extremely resource-efficient for problems of this size. Because the model is stochastic, accuracy is reported as a confidence interval: repeating the baseline across 30 independent seeds yields a mean of 18.1 failures with a 95% confidence half-width of ± 1.24 , and a mean of 24.4 cascade events (± 7.1). These intervals quantify the uncertainty in any single run and confirm that the qualitative conclusions are statistically robust rather than artefacts of one random sequence (Law, 2015).

Task 3: Tool and Technique Comparison

3.1 Comparison of Two Simulation Tools/Techniques

Two simulation approaches were evaluated for this scenario: Python with SimPy (the technique adopted) and AnyLogic, a commercial multi-method simulation platform (Borshchev and Filippov, 2004). Comparing them clarifies why the chosen tool is appropriate and where alternatives might excel.

SimPy is an open-source, process-based discrete-event library embedded in general-purpose Python. Its capabilities centre on lightweight generator-based processes, shared resources with

queueing, and event scheduling, all expressed in ordinary Python code. Its principal advantages are that it is free, fully scriptable, version-controllable and integrates seamlessly with the scientific Python stack — NumPy, pandas, Matplotlib and ipywidgets — which made the entire analysis, visualisation and Colab interface achievable in a single ecosystem. Its limitations are equally clear: it offers no graphical model editor, no built-in animation, and no native continuous or agent-based engines, so any such behaviour must be hand-coded. For a transparent, reproducible discrete-event model of modest size, however, these are minor constraints, and its suitability for the present cascading-failure problem is high.

AnyLogic, by contrast, is a mature commercial platform that uniquely supports discrete-event, agent-based and system-dynamics modelling within one environment, together with a drag-and-drop graphical editor, rich 2D/3D animation and geographic-information-system integration (Borshchev and Filippov, 2004). For a smart-grid study these features are genuinely attractive: agent-based representation of individual consumers or generators, continuous power-flow dynamics, and built-in spatial visualisation could all be combined without leaving the tool. Its disadvantages are cost — the professional edition is expensive and the free Personal Learning Edition restricts model size and export — and a degree of vendor lock-in, since models are stored in a proprietary format that is harder to version-control or embed in a free cloud notebook than plain Python.

The learning curve and resource requirements differ markedly. SimPy demands existing Python proficiency but almost no tool-specific learning, runs on any machine and in free cloud environments such as Colab, and has negligible computational footprint. AnyLogic requires learning a substantial graphical environment and some Java, and benefits from a more capable workstation, but in return reduces the amount of code that must be written for complex, multi-method models. For this assignment — where transparency, reproducibility, zero cost and easy cloud execution were paramount, and where the dynamics are predominantly discrete — SimPy was the more suitable choice; AnyLogic would become preferable if the study were extended to large-scale, multi-method models requiring polished stakeholder-facing animation.

3.2 Alternative Approaches and Reflection

Alternative approaches would have illuminated different facets of the problem. A system-dynamics model built around stocks and flows (Sterman, 2000) would have represented aggregate generation, demand and reserve margins as smoothly varying quantities and would

have been well suited to studying long-run feedback such as investment cycles or gradual demand growth. It would, however, have obscured the very mechanism this study targets: system dynamics averages over individual components and so cannot naturally represent a single line tripping and forcing a discrete redistribution onto its neighbours. The cascading-failure signature that emerges so clearly from the discrete-event model would have been smoothed away.

An agent-based model would have produced complementary insights by endowing each generator, line and consumer with autonomous behavioural rules and letting system behaviour emerge from their interactions (Macal and North, 2010). This would be powerful for studying heterogeneous consumer responses, distributed control or market behaviour, but at the cost of greater implementation effort and more parameters to calibrate and validate. The chosen discrete-event approach occupies a deliberate middle ground: it captures individual component failure and resource contention — the essential ingredients of cascading failure — while remaining simple, fast and transparent.

The principal strength of the adopted approach is therefore its clarity and efficiency: the model is small enough to be fully understood and audited, runs in milliseconds, and isolates the cascade mechanism cleanly. Its main weakness is physical fidelity — capacity-proportional redistribution is a simplification of true power-flow physics, and the failure model is probabilistic rather than derived from first principles. A natural and valuable extension would be a hybrid model coupling the discrete-event engine to a proper AC power-flow solver, retaining the present clarity while improving electrical realism.

Task 4: Business Value and Future Directions

4.1 Competitive Advantage and Return on Investment

For a distribution system operator, a simulation of this kind translates directly into competitive and decision-making advantage. By quantifying how additional repair crews, reduced peak loading or improved asset condition change failure and cascade counts, the model supports evidence-based capital-allocation and maintenance decisions rather than reliance on rules of thumb. The scenario results make the trade-offs explicit: the marginal benefit of a third repair crew, for example, can be weighed against its cost in reduced outage duration and avoided unserved energy.

The cost-benefit case is compelling because the tool itself is almost free. Built on open-source software and runnable in a free cloud notebook, its development and operating costs are negligible, whereas the losses it helps avoid are large: major blackouts impose economic costs running into billions (U.S.-Canada Power System Outage Task Force, 2004). Even a small reduction in outage frequency or duration, or a better-targeted maintenance budget, yields a return on investment far exceeding the modelling effort. The same model also supports regulatory reporting and resilience planning, broadening its organisational value beyond operations into strategy and compliance.

4.2 Emerging Trends and Ethical Considerations

Several emerging trends could substantially enhance this simulation. The most significant is the digital twin: a continuously updated virtual replica of the physical grid, fed by live sensor data, that mirrors the real system in real time (Tao et al., 2019). Coupling the present model to live SCADA feeds would turn it from an offline planning tool into an online early-warning system. Artificial-intelligence integration is a natural complement — machine-learning models could forecast demand and renewable output, learn failure probabilities from historical data, and even recommend optimal repair-crew dispatch — while cloud-based simulation, already demonstrated by the Colab deployment, would allow thousands of scenarios to be run in parallel for probabilistic risk assessment.

These enhancements carry ethical responsibilities. A model used to guide investment and resilience decisions must be transparent and rigorously validated, because over-confidence in a simplified simulation could lead operators to under-resource a genuinely vulnerable grid, with public-safety consequences. Connecting a digital twin to live infrastructure data raises cybersecurity and privacy concerns: the data stream is itself a critical asset and an attack surface, and consumption data can be personally revealing. There is also an equity dimension — resilience investments guided by the model should not systematically privilege wealthier zones over vulnerable communities. Responsible deployment therefore requires clear communication of model assumptions and uncertainty, robust data governance, and human oversight of any automated decisions (Rinaldi, Peerenboom and Kelly, 2001).

References

- AEMO (2017) Black System South Australia 28 September 2016: Final Report. Melbourne: Australian Energy Market Operator.
- Amin, S.M. and Wollenberg, B.F. (2005) 'Toward a smart grid: power delivery for the 21st century', *IEEE Power and Energy Magazine*, 3(5), pp. 34–41.
- Banks, J., Carson, J.S., Nelson, B.L. and Nicol, D.M. (2010) *Discrete-Event System Simulation*. 5th edn. Upper Saddle River, NJ: Pearson.
- Borshchev, A. and Filippov, A. (2004) 'From system dynamics and discrete event to practical agent based modeling: reasons, techniques, tools', *Proceedings of the 22nd International Conference of the System Dynamics Society*, Oxford, 25–29 July.
- Buldyrev, S.V., Parshani, R., Paul, G., Stanley, H.E. and Havlin, S. (2010) 'Catastrophic cascade of failures in interdependent networks', *Nature*, 464(7291), pp. 1025–1028.
- Dobson, I., Carreras, B.A., Lynch, V.E. and Newman, D.E. (2007) 'Complex systems analysis of series of blackouts: cascading failure, critical points, and self-organization', *Chaos*, 17(2), 026103.
- ENTSO-E (2021) Continental Europe Synchronous Area Separation on 8 January 2021: Final Report. Brussels: European Network of Transmission System Operators for Electricity.
- Kundur, P., Paserba, J., Ajarapu, V., Andersson, G., Bose, A., Canizares, C., Hatziaargyriou, N., Hill, D., Stankovic, A., Taylor, C., Van Cutsem, T. and Vittal, V. (2004) 'Definition and classification of power system stability', *IEEE Transactions on Power Systems*, 19(3), pp. 1387–1401.
- Law, A.M. (2015) *Simulation Modeling and Analysis*. 5th edn. New York: McGraw-Hill.
- Macal, C.M. and North, M.J. (2010) 'Tutorial on agent-based modelling and simulation', *Journal of Simulation*, 4(3), pp. 151–162.
- Panteli, M. and Mancarella, P. (2015) 'Influence of extreme weather and climate change on the resilience of power systems', *IEEE Systems Journal*, 9(3), pp. 1733–1742.
- Rinaldi, S.M., Peerenboom, J.P. and Kelly, T.K. (2001) 'Identifying, understanding, and analyzing critical infrastructure interdependencies', *IEEE Control Systems Magazine*, 21(6), pp. 11–25.
- Sterman, J.D. (2000) *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Boston: Irwin/McGraw-Hill.
- Tao, F., Zhang, H., Liu, A. and Nee, A.Y.C. (2019) 'Digital twin in industry: state-of-the-art', *IEEE Transactions on Industrial Informatics*, 15(4), pp. 2405–2415.
- Team SimPy (2023) *SimPy Discrete-Event Simulation for Python: Documentation*. Available at: <https://simpy.readthedocs.io> (Accessed: 2 June 2026).

U.S.-Canada Power System Outage Task Force (2004) Final Report on the August 14, 2003
Blackout in the United States and Canada: Causes and Recommendations. Washington,
DC / Ottawa.

Appendix A: Task 2 Source Code

The complete, runnable source code for the discrete-event simulation is reproduced below and is also hosted at <https://github.com/khadkasaleena/smart-grid-simulation>, where it can be executed directly in Google Colab via the badge in the README.

config.py

```
"""
config.py
Configuration and system structure for the Smart Grid Cascading Failure simulation.

All tunable parameters live here so the model, the scenario runner and the
Colab user interface can share a single source of truth. Scenarios override
individual scalar values via a plain dict (see scenarios.py).

Module: Advanced Simulation Techniques (Task 2)
Author: Salina Khadka, BSBI Berlin
"""

# -----
# Physical grid structure (Task 2.1 – entities and relationships)
# -----

# Generators and their rated capacity in MW.
# Coal units are dispatchable baseload; the solar farm follows daylight.
GENERATORS = {
    "Coal Gen 1": {"capacity": 300, "type": "coal", "dispatchable": True},
    "Coal Gen 2": {"capacity": 200, "type": "coal", "dispatchable": True},
    "Solar Farm": {"capacity": 150, "type": "solar", "dispatchable": False},
}

# Transmission lines with rated thermal capacity in MW.
LINE_CAPACITIES = {
    "L1-A": 250, # Coal Gen 1 -> Substation A
    "L2-A": 200, # Coal Gen 2 -> Substation A
    "L3-B": 180, # Solar Farm -> Substation B
    "L4-C": 220, # Substation A -> Substation C
    "L5-D": 160, # Substation B -> Substation D
}

# Substations and the share of base demand drawn by their consumer zone.
SUBSTATIONS = {
    "Substation A": {"zone": "Zone 1: Industrial", "base_load_frac": 0.40},
    "Substation B": {"zone": "Zone 2: Residential", "base_load_frac": 0.25},
    "Substation C": {"zone": "Zone 3: Commercial", "base_load_frac": 0.20},
    "Substation D": {"zone": "Zone 4: Mixed", "base_load_frac": 0.15},
}

# Total installed generation capacity (MW).
TOTAL_CAPACITY = sum(g["capacity"] for g in GENERATORS.values()) # 650 MW

# -----
# Default run parameters (the baseline scenario)
# -----

DEFAULT_CONFIG = {
    "sim_duration": 72, # simulation horizon in hours
    "time_step": 1, # advance one simulated hour per step
    "base_fail_prob": 0.05, # per-line failure probability per hour at normal load
```

```

    "cascade_thresh": 1.20, # >120% of rated capacity triggers a cascade
    "overload_fail_mult": 5.0, # failure-probability multiplier when overloaded
    "repair_time_min": 2.0, # minimum repair duration (sim hours)
    "repair_time_max": 4.0, # maximum repair duration (sim hours)
    "num_repair_crews": 1, # repair crews available (a SimPy resource)
    "demand_multiplier": 1.0, # global scaling of demand (scenario lever)
    "solar_scale": 1.0, # global scaling of solar output (scenario lever)
    "seed": 42, # RNG seed for reproducibility
}

```

```

def make_config(**overrides):
    """Return a fresh copy of DEFAULT_CONFIG with the given keys overridden."""
    cfg = dict(DEFAULT_CONFIG)
    cfg.update(overrides)
    return cfg

```

grid_model.py

```

"""
grid_model.py
The discrete-event SimPy model of the smart grid (Task 2.2).

```

The whole simulation is wrapped in a GridSimulation class so that each run owns its own random-number generator and state. This isolation is what makes the multi-scenario sweep in scenarios.py reproducible – no global state leaks between runs.

Concurrent SimPy processes:

- * generator_process – sets generator output every hour
- * demand_process – sets total demand every hour (time-of-day profile)
- * line_monitor – one per transmission line: checks overload/failure
- * repair_process – spawned on failure; competes for the repair crew
- * metrics_collector – samples the system state every hour for plotting

Module: Advanced Simulation Techniques (Task 2)

Author: Salina Khadka, BSBI Berlin

```

"""
import math
import random
import time as wall_clock

import simpy

from config import (
    GENERATORS,
    LINE_CAPACITIES,
    TOTAL_CAPACITY,
    make_config,
)

class GridSimulation:
    """A single, self-contained run of the smart-grid cascading-failure model."""

    def __init__(self, config=None, seed=None):
        # Merge caller overrides onto the defaults.
        self.cfg = make_config(*(config or {}))
        if seed is not None:
            self.cfg["seed"] = seed

        # Dedicated RNG -> reproducible and isolated from other runs.

```

```

self.rng = random.Random(self.cfg["seed"])

# --- Mutable grid state -----
self.gen_output = {name: 0.0 for name in GENERATORS}
self.line_loads = {lid: 0.0 for lid in LINE_CAPACITIES}
self.line_status = {lid: True for lid in LINE_CAPACITIES} # True = in service
self.demand = 0.0

# --- Counters -----
self.failures = 0
self.recoveries = 0
self.cascade_events = 0
self.repair_times = [] # actual outage durations (sim hours)
self.crew_busy_hours = 0.0 # for repair-crew utilisation
self.event_log = [] # (time, type, message)

# --- Time-series for plotting / export -----
self.ts = {
    "time": [], "coal1": [], "coal2": [], "solar": [],
    "demand": [], "load_pct": [], "lines_up": [],
}
# Per-line load% history -> used for the heatmap (Task 2.3).
self.ts_line_pct = {lid: [] for lid in LINE_CAPACITIES}
self.fail_times = [] # sim hours at which failures occur

# ----- #
# Logging
# ----- #
def log(self, env, etype, msg):
    self.event_log.append((env.now, etype, msg))

# ----- #
# Demand and solar profiles
# ----- #
def demand_fraction(self, hour):
    """Demand as a fraction of TOTAL_CAPACITY, by time of day (+/-5% jitter)."""
    h = hour % 24
    if 23 <= h or h < 6:
        base = 0.60 # overnight trough
    elif (6 <= h < 8) or (20 <= h < 23):
        base = 0.85 # shoulder
    elif (8 <= h < 10) or (17 <= h < 20):
        base = 1.10 # morning / evening peak
    else:
        base = 0.90 # daytime standard
    jitter = self.rng.uniform(-0.05, 0.05)
    return (base + jitter) * self.cfg["demand_multiplier"]

def solar_output(self, hour, capacity):
    """Sinusoidal daylight curve scaled by random cloud cover."""
    h = hour % 24
    if 6.0 <= h <= 20.0:
        sun = math.sin(math.pi * (h - 6) / 14)
        cloud = self.rng.uniform(0.55, 1.0)
        return capacity * sun * cloud * self.cfg["solar_scale"]
    return 0.0

# ----- #
# Load distribution
# ----- #
def distribute_load(self):
    """Spread current demand across in-service lines, proportional to capacity."""
    active = [lid for lid, up in self.line_status.items() if up]
    if not active:

```

```

        for lid in LINE_CAPACITIES:
            self.line_loads[lid] = 0.0
        return
    total_active_cap = sum(LINE_CAPACITIES[lid] for lid in active)
    for lid in LINE_CAPACITIES:
        if self.line_status[lid]:
            share = LINE_CAPACITIES[lid] / total_active_cap
            self.line_loads[lid] = self.demand * share
        else:
            self.line_loads[lid] = 0.0

# ----- #
# SimPy processes
# ----- #
def generator_process(self, env):
    c1 = GENERATORS["Coal Gen 1"]["capacity"]
    c2 = GENERATORS["Coal Gen 2"]["capacity"]
    cs = GENERATORS["Solar Farm"]["capacity"]
    while True:
        self.gen_output["Coal Gen 1"] = c1
        self.gen_output["Coal Gen 2"] = c2
        self.gen_output["Solar Farm"] = self.solar_output(env.now, cs)
        yield env.timeout(self.cfg["time_step"])

def demand_process(self, env):
    while True:
        self.demand = TOTAL_CAPACITY * self.demand_fraction(env.now)
        yield env.timeout(self.cfg["time_step"])

def repair_process(self, env, line_id, fail_time):
    """Wait for a free crew, repair the line, restore service."""
    with self.repair_crew.request() as req:
        yield req
        duration = self.rng.uniform(self.cfg["repair_time_min"],
                                     self.cfg["repair_time_max"])
        self.crew_busy_hours += duration
        yield env.timeout(duration)

        self.line_status[line_id] = True
        self.recoveries += 1
        self.repair_times.append(env.now - fail_time)
        self.distribute_load()
        self.log(env, "RESTORE",
                 f"{line_id} restored after {env.now - fail_time:.2f} h "
                 f"(crew time {duration:.2f} h)")

def line_monitor(self, env, line_id):
    """Per-line hourly check for overload and stochastic failure."""
    cap = LINE_CAPACITIES[line_id]
    while True:
        if self.line_status[line_id]:
            load = self.line_loads[line_id]
            frac = load / cap

            if frac > self.cfg["cascade_thresh"]:
                self.cascade_events += 1
                self.log(env, "CASCADE",
                         f"{line_id} OVERLOADED at {frac*100:.1f}% "
                         f"({load:.1f}/{cap} MW)")
                fail_prob = min(self.cfg["base_fail_prob"]
                                * self.cfg["overload_fail_mult"], 0.90)
            else:
                fail_prob = self.cfg["base_fail_prob"]

```

```

        if self.rng.random() < fail_prob:
            self.line_status[line_id] = False
            self.failures += 1
            self.fail_times.append(env.now)
            self.log(env, "FAILURE",
                    f"{line_id} FAILED (load {load:.1f}/{cap} MW, "
                    f"{frac*100:.1f}%)"
            self.distribute_load()

            # Did the redistribution overload a neighbour? (cascade)
            for other in LINE_CAPACITIES:
                if other != line_id and self.line_status[other]:
                    of = self.line_loads[other] / LINE_CAPACITIES[other]
                    if of > self.cfg["cascade_thresh"]:
                        self.cascade_events += 1
                        self.log(env, "CASCADE",
                                f" -> {other} now {of*100:.1f}% "
                                f"after redistribution")

            env.process(self.repair_process(env, line_id, env.now))

    yield env.timeout(self.cfg["time_step"])

def metrics_collector(self, env):
    """Sample state once per hour for the time-series outputs."""
    while True:
        self.ts["time"].append(env.now)
        self.ts["coal1"].append(self.gen_output["Coal Gen 1"])
        self.ts["coal2"].append(self.gen_output["Coal Gen 2"])
        self.ts["solar"].append(self.gen_output["Solar Farm"])
        self.ts["demand"].append(self.demand)

        active = [lid for lid, up in self.line_status.items() if up]
        if active:
            avg_pct = sum(self.line_loads[lid] / LINE_CAPACITIES[lid]
                          for lid in active) / len(active) * 100
        else:
            avg_pct = 0.0
        self.ts["load_pct"].append(avg_pct)
        self.ts["lines_up"].append(len(active))

        for lid in LINE_CAPACITIES:
            pct = (self.line_loads[lid] / LINE_CAPACITIES[lid] * 100
                  if self.line_status[lid] else 0.0)
            self.ts_line_pct[lid].append(pct)

    yield env.timeout(self.cfg["time_step"])

# ----- #
# Runner
# ----- #
def run(self):
    """Execute the simulation and return a results dictionary."""
    env = simpy.Environment()
    self.repair_crew = simpy.Resource(env, capacity=self.cfg["num_repair_crews"])

    env.process(self.generator_process(env))
    env.process(self.demand_process(env))
    env.process(self.metrics_collector(env))
    for lid in LINE_CAPACITIES:
        env.process(self.line_monitor(env, lid))

    # Prime the first hour so monitors see a sensible load.
    self.demand = TOTAL_CAPACITY * self.demand_fraction(0)

```

```

        self.distribute_load()

        t0 = wall_clock.perf_counter()
        env.run(until=self.cfg["sim_duration"])
        exec_ms = (wall_clock.perf_counter() - t0) * 1000.0

        mttr = (sum(self.repair_times) / len(self.repair_times)
                 if self.repair_times else 0.0)
        peak_pct = max(self.ts["load_pct"]) if self.ts["load_pct"] else 0.0
        crew_util = (self.crew_busy_hours
                     / (self.cfg["num_repair_crews"] * self.cfg["sim_duration"]))

    return {
        "config": self.cfg,
        "exec_ms": exec_ms,
        "failures": self.failures,
        "recoveries": self.recoveries,
        "cascade_events": self.cascade_events,
        "mttr": mttr,
        "peak_load_pct": peak_pct,
        "crew_utilisation": crew_util,
        "unrecovered": self.failures - self.recoveries,
    }

def run_once(config=None, seed=None):
    """Convenience helper: build, run, and return (sim, results)."""
    sim = GridSimulation(config=config, seed=seed)
    results = sim.run()
    return sim, results

```

metrics.py

"""

metrics.py

Performance and accuracy metrics for the simulation (Task 2.4).

Provides:

- * summarise() – pretty one-line dict of headline metrics for a run
- * accuracy_study() – repeat the same scenario across many seeds and report the mean and a 95% confidence half-width. Because the model is stochastic, this confidence interval is our "accuracy measure": it tells us how trustworthy a single run's numbers are.

Module: Advanced Simulation Techniques (Task 2)

Author: Salina Khadka, BSBI Berlin

"""

import math

from grid_model import GridSimulation

```

def summarise(results):
    """Return a flat dict of the headline metrics for one run."""
    return {
        "failures": results["failures"],
        "recoveries": results["recoveries"],
        "unrecovered": results["unrecovered"],
        "cascade_events": results["cascade_events"],
        "mttr_h": round(results["mttr"], 2),
        "peak_load_pct": round(results["peak_load_pct"], 1),
    }

```

```

        "crew_utilisation": round(results["crew_utilisation"], 3),
        "exec_ms": round(results["exec_ms"], 2),
    }

def _mean_std(values):
    n = len(values)
    mean = sum(values) / n
    if n < 2:
        return mean, 0.0
    var = sum((v - mean) ** 2 for v in values) / (n - 1)
    return mean, math.sqrt(var)

def accuracy_study(config=None, n_runs=30, base_seed=1000):
    """
    Run the same configuration with `n_runs` different seeds and report the
    mean and 95% confidence half-width for the key output metrics.

    Returns a dict: metric -> {"mean", "std", "ci95", "n"}.
    The 95% CI half-width is 1.96 * std / sqrt(n).
    """
    collected = {"failures": [], "cascade_events": [], "mttr": [],
                 "peak_load_pct": [], "crew_utilisation": []}

    for i in range(n_runs):
        sim = GridSimulation(config=config, seed=base_seed + i)
        r = sim.run()
        collected["failures"].append(r["failures"])
        collected["cascade_events"].append(r["cascade_events"])
        collected["mttr"].append(r["mttr"])
        collected["peak_load_pct"].append(r["peak_load_pct"])
        collected["crew_utilisation"].append(r["crew_utilisation"])

    report = {}
    for metric, values in collected.items():
        mean, std = _mean_std(values)
        ci95 = 1.96 * std / math.sqrt(n_runs) if n_runs > 1 else 0.0
        report[metric] = {"mean": mean, "std": std, "ci95": ci95, "n": n_runs}
    return report

def print_accuracy_report(report):
    print("\nAccuracy study (mean +/- 95% CI across "
          f"{next(iter(report.values()))['n']} seeds)")
    print("-" * 56)
    for metric, s in report.items():
        print(f" {metric:18s}: {s['mean']:8.3f} +/- {s['ci95']:8.3f}")
    print("-" * 56)

```

scenarios.py

```

"""
scenarios.py
Defines and runs the alternative "what-if" scenarios (Task 2.4).

```

Each scenario is just a name plus a dict of config overrides. Running them all through the same model and comparing the headline metrics is what lets us study how demand growth, ageing infrastructure or extra repair crews change the grid's resilience.

Module: Advanced Simulation Techniques (Task 2)
 Author: Salina Khadka, BSBI Berlin

```

"""

from grid_model import GridSimulation
from metrics import summarise

# name -> (description, config overrides)
SCENARIOS = {
    "Baseline": (
        "Normal demand, single repair crew",
        {},
    ),
    "High Demand": (
        "Demand +25% (e.g. heatwave / load growth)",
        {"demand_multiplier": 1.25},
    ),
    "Aged Infrastructure": (
        "Failure probability tripled (ageing assets)",
        {"base_fail_prob": 0.15},
    ),
    "High Renewable": (
        "Solar capacity factor boosted, demand normal",
        {"solar_scale": 1.4},
    ),
    "Extra Repair Crews": (
        "Three repair crews instead of one",
        {"num_repair_crews": 3},
    ),
    "Stress Test": (
        "High demand AND aged infrastructure together",
        {"demand_multiplier": 1.25, "base_fail_prob": 0.15},
    ),
}

def run_scenario(name, seed=42):
    """Run a single named scenario and return its summarised metrics."""
    if name not in SCENARIOS:
        raise KeyError(f"Unknown scenario: {name}")
    _desc, overrides = SCENARIOS[name]
    sim = GridSimulation(config=overrides, seed=seed)
    results = sim.run()
    return sim, results

def run_all(seed=42):
    """Run every scenario and return an ordered list of summary rows."""
    rows = []
    for name, (desc, overrides) in SCENARIOS.items():
        sim = GridSimulation(config=overrides, seed=seed)
        results = sim.run()
        row = {"scenario": name, "description": desc}
        row.update(summarise(results))
        rows.append(row)
    return rows

def print_comparison(rows):
    """Print a tidy comparison table of all scenarios."""
    cols = ["scenario", "failures", "cascade_events", "mttr_h",
            "peak_load_pct", "crew_utilisation", "unrecovered", "exec_ms"]
    widths = {c: max(len(c), max(len(str(r[c])) for r in rows)) for c in cols}
    header = " ".join(c.ljust(widths[c]) for c in cols)
    print(header)

```



```

print("-" * len(header))
for r in rows:
    print(" ".join(str(r[c]).ljust(widths[c]) for c in cols))

def export_comparison_csv(rows, path="scenario_comparison.csv"):
    import csv
    cols = ["scenario", "description", "failures", "recoveries", "unrecovered",
            "cascade_events", "mttr_h", "peak_load_pct", "crew_utilisation",
            "exec_ms"]
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for r in rows:
            w.writerow({c: r.get(c, "") for c in cols})
    return path

```

visualization.py

```

"""
visualization.py
Plots, heatmap and CSV export for the simulation (Task 2.3).

```

Every function takes a **finished** GridSimulation object (its time-series are already populated) and either returns a matplotlib Figure (so the Colab UI can display it inline) or writes a file. Keeping the functions Figure-returning means the same code works headless (saved PNG) and interactively (Colab).

```

Module: Advanced Simulation Techniques (Task 2)
Author: Salina Khadka, BSBI Berlin
"""

```

```

import csv

import numpy as np
import matplotlib.pyplot as plt

from config import LINE_CAPACITIES, TOTAL_CAPACITY

# ----- #
# 1. Generation vs demand
# ----- #
def plot_power_output(sim):
    t = np.array(sim.ts["time"])
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.stackplot(t, sim.ts["coal1"], sim.ts["coal2"], sim.ts["solar"],
                 labels=["Coal Gen 1 (300 MW)", "Coal Gen 2 (200 MW)",
                        "Solar Farm (150 MW)"],
                 colors=["#4a4a8a", "#7a7ac0", "#f5a623"], alpha=0.85)
    ax.plot(t, sim.ts["demand"], color="#e74c3c", linewidth=2.0,
            linestyle="--", label="Total Demand (MW)")
    ax.set_xlabel("Simulation Time (hours)")
    ax.set_ylabel("Power (MW)")
    ax.set_title("Generator Power Output vs Demand", fontweight="bold")
    ax.legend(loc="upper right", fontsize=9)
    ax.set_xlim(0, sim.cfg["sim_duration"])
    ax.set_ylim(0, TOTAL_CAPACITY * 1.25)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    return fig

```

```

# ----- #
# 2. Average line load % with overload thresholds
# ----- #
def plot_grid_load(sim):
    t = np.array(sim.ts["time"])
    load = np.array(sim.ts["load_pct"])
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.fill_between(t, load, alpha=0.3, color="#2980b9")
    ax.plot(t, load, color="#2980b9", linewidth=1.5, label="Average line load %")
    ax.axhline(100, color="#e67e22", linewidth=1.5, linestyle="--",
              label="100% rated capacity")
    ax.axhline(sim.cfg["cascade_thresh"] * 100, color="#e74c3c",
              linewidth=2.0, linestyle="-. ", label="Cascade threshold")
    for ft in sim.fail_times:
        ax.axvline(ft, color="#c0392b", linewidth=0.8, alpha=0.5)
    ax.set_xlabel("Simulation Time (hours)")
    ax.set_ylabel("Average Line Load (%)")
    ax.set_title("Grid Load Percentage Over Time", fontweight="bold")
    ax.legend(fontsize=9, loc="upper right")
    ax.set_xlim(0, sim.cfg["sim_duration"])
    ax.set_ylim(0, 160)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    return fig

# ----- #
# 3. Failure-events timeline
# ----- #
def plot_failures(sim):
    fig, ax = plt.subplots(figsize=(12, 3.5))
    if sim.fail_times:
        ml, sl, bl = ax.stem(sim.fail_times, [1] * len(sim.fail_times),
                           linefmt="#e74c3c", markerfmt="D", basefmt=" ")
        ml.set_markerfacecolor("#e74c3c")
        ml.set_markersize(8)
    ax.set_xlabel("Simulation Time (hours)")
    ax.set_title("Transmission Line Failure Events Timeline", fontweight="bold")
    ax.set_xlim(0, sim.cfg["sim_duration"])
    ax.set_ylim(0, 2)
    ax.set_yticks([])
    ax.grid(True, alpha=0.2, axis="x")
    fig.tight_layout()
    return fig

# ----- #
# 4. Heatmap – per-line load % over time (Task 2.3 asks for heatmaps)
# ----- #
def plot_line_heatmap(sim):
    line_ids = list(LINE_CAPACITIES.keys())
    matrix = np.array([sim.ts_line_pct[lid] for lid in line_ids]) # rows=lines

    fig, ax = plt.subplots(figsize=(12, 4))
    im = ax.imshow(matrix, aspect="auto", cmap="RdYlGn_r",
                  vmin=0, vmax=140, origin="lower",
                  extent=[0, sim.cfg["sim_duration"], -0.5, len(line_ids) - 0.5])
    ax.set_yticks(range(len(line_ids)))
    ax.set_yticklabels(line_ids)
    ax.set_xlabel("Simulation Time (hours)")
    ax.set_ylabel("Transmission Line")
    ax.set_title("Per-Line Load Heatmap (% of rated capacity)", fontweight="bold")
    cbar = fig.colorbar(im, ax=ax, pad=0.01)
    cbar.set_label("Load (% of capacity)")

```

```

fig.tight_layout()
return fig

# ----- #
# Saving helpers
# ----- #
def save_all_figures(sim, prefix="graph_", outdir="."):
    """Save the four standard figures as PNGs. Returns the list of paths."""
    import os
    builders = {
        "power_output": plot_power_output,
        "grid_load": plot_grid_load,
        "failures": plot_failures,
        "heatmap": plot_line_heatmap,
    }
    paths = []
    for name, builder in builders.items():
        fig = builder(sim)
        path = os.path.join(outdir, f"{prefix}{name}.png")
        fig.savefig(path, dpi=150)
        plt.close(fig)
        paths.append(path)
    return paths

# ----- #
# CSV export (Task 2.3 – "export results for further analysis")
# ----- #
def export_timeseries_csv(sim, path="simulation_results.csv"):
    """Write the hourly time-series to a CSV file."""
    line_ids = list(LINE_CAPACITIES.keys())
    header = ([ "hour", "demand_MW", "coal1_MW", "coal2_MW", "solar_MW",
        "avg_load_pct", "lines_in_service"
        + [f"{lid}_load_pct" for lid in line_ids]
    ])
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(header)
        for i, hour in enumerate(sim.ts["time"]):
            row = [hour,
                round(sim.ts["demand"][i], 2),
                round(sim.ts["coal1"][i], 2),
                round(sim.ts["coal2"][i], 2),
                round(sim.ts["solar"][i], 2),
                round(sim.ts["load_pct"][i], 2),
                sim.ts["lines_up"][i]]
            row += [round(sim.ts_line_pct[lid][i], 2) for lid in line_ids]
            w.writerow(row)
    return path

def export_event_log_csv(sim, path="event_log.csv"):
    """Write the discrete event log (failures, cascades, restores) to CSV."""
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["sim_hour", "event_type", "description"])
        for t, etype, msg in sim.event_log:
            w.writerow([round(t, 2), etype, msg])
    return path

```

make_system_diagram.py

```
"""
make_system_diagram.py
Generates the conceptual system diagram for Task 2.1.

Draws the grid topology (generators -> lines -> substations -> consumer zones)
together with the simulation's control loop (monitor -> repair crew). Saves
system_diagram.png. Pure matplotlib so it runs anywhere, including Colab.

Module: Advanced Simulation Techniques (Task 2)
Author: Salina Khadka, BSBI Berlin
"""

import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

def box(ax, x, y, w, h, text, color):
    ax.add_patch(FancyBboxPatch((x, y), w, h,
                               boxstyle="round,pad=0.02,rounding_size=0.06",
                               linewidth=1.2, edgecolor="#444444",
                               facecolor=color))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
           fontsize=9, fontweight="bold", wrap=True)

def arrow(ax, x1, y1, x2, y2, style="-|>", color="#333333", ls="-"):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle=style,
                                mutation_scale=14, linewidth=1.3,
                                color=color, linestyle=ls,
                                connectionstyle="arc3,rad=0.0"))

def main():
    fig, ax = plt.subplots(figsize=(13, 8))
    ax.set_xlim(0, 13)
    ax.set_ylim(0, 8)
    ax.axis("off")

    gen_c = "#cdd7f0"    # generators
    line_c = "#fde2b5"    # transmission
    sub_c = "#cfe9d4"     # substations
    zone_c = "#e7d4f0"    # consumer zones
    ctrl_c = "#f5c6c6"    # control / repair

    ax.text(6.5, 7.7, "Smart Grid Cascading-Failure Simulation – System Diagram",
           ha="center", fontsize=13, fontweight="bold")

    # Column headers
    for x, label in [(1.4, "GENERATION"), (4.2, "TRANSMISSION"),
                    (7.4, "SUBSTATIONS"), (10.6, "CONSUMER ZONES")]:
        ax.text(x, 7.1, label, ha="center", fontsize=9.5,
               fontweight="bold", color="#555555")

    # Generators
    gens = [("Coal Gen 1\n300 MW", 6.0), ("Coal Gen 2\n200 MW", 4.3),
            ("Solar Farm\n150 MW", 2.6)]
    for txt, y in gens:
        box(ax, 0.4, y, 2.0, 1.1, txt, gen_c)

    # Transmission lines
    lines = [("L1-A 250 MW", 6.0), ("L2-A 200 MW", 4.3),
            ("L3-B 180 MW", 2.6), ("L4-C 220 MW", 1.4),
```

```

        ("L5-D 160 MW", 0.2)]
    for txt, y in lines:
        box(ax, 3.3, y, 1.8, 0.75, txt, line_c)

    # Substations
    subs = [("Substation A", 6.0), ("Substation B", 4.0),
            ("Substation C", 2.2), ("Substation D", 0.5)]
    for txt, y in subs:
        box(ax, 6.5, y, 1.9, 0.9, txt, sub_c)

    # Consumer zones
    zones = [("Zone 1\nIndustrial (40%)", 6.0), ("Zone 2\nResidential (25%)", 4.0),
            ("Zone 3\nCommercial (20%)", 2.2), ("Zone 4\nMixed (15%)", 0.5)]
    for txt, y in zones:
        box(ax, 9.7, y, 2.0, 0.9, txt, zone_c)

    # Generation -> transmission
    arrow(ax, 2.4, 6.55, 3.3, 6.35) # coal1 -> L1
    arrow(ax, 2.4, 4.85, 3.3, 4.65) # coal2 -> L2
    arrow(ax, 2.4, 3.15, 3.3, 2.95) # solar -> L3

    # Transmission -> substations
    arrow(ax, 5.1, 6.35, 6.5, 6.45) # L1 -> A
    arrow(ax, 5.1, 4.65, 6.5, 6.2) # L2 -> A
    arrow(ax, 5.1, 2.95, 6.5, 4.45) # L3 -> B
    arrow(ax, 5.1, 1.75, 6.5, 2.65) # L4 -> C
    arrow(ax, 5.1, 0.55, 6.5, 0.95) # L5 -> D

    # Substations -> zones
    for ys, yz in [(6.45, 6.45), (4.45, 4.45), (2.65, 2.65), (0.95, 0.95)]:
        arrow(ax, 8.4, ys, 9.7, yz)

    # Control loop: repair crew (dashed feedback to failed lines)
    box(ax, 0.4, 0.1, 2.0, 0.9, "Repair Crew\n(SimPy Resource)", ctrl_c)
    arrow(ax, 3.3, 0.45, 2.4, 0.55, style="-|>", color="#c0392b", ls="--")
    ax.text(2.85, 0.15, "repair", color="#c0392b", fontsize=8, ha="center")

    # Boundary box (system boundary)
    ax.add_patch(plt.Rectangle((0.2, -0.2), 11.7, 7.0, fill=False,
                               linestyle="--", linewidth=1.3, edgecolor="#888888"))
    ax.text(0.3, 6.9, "System boundary", fontsize=8.5, color="#888888",
           style="italic")

    fig.tight_layout()
    fig.savefig("system_diagram.png", dpi=150, bbox_inches="tight")
    print("Saved: system_diagram.png")

if __name__ == "__main__":
    main()

```

run_simulation.py

```

"""
run_simulation.py
Command-line entry point that ties everything together.

```

Running `python run\_simulation.py` will:

1. run the baseline scenario and print its metrics,
2. save the four figures (power, load, failures, heatmap) as PNGs,
3. export the time-series and event log to CSV,
4. run every what-if scenario and print a comparison table,
5. run a multi-seed accuracy study and print mean +/- 95% CI.

Module: Advanced Simulation Techniques (Task 2)

Author: Salina Khadka, BSBI Berlin

"""

```
import os
```

```
import matplotlib
```

```
matplotlib.use("Agg") # headless backend for the CLI run
```

```
from grid_model import GridSimulation
```

```
from metrics import summarise, accuracy_study, print_accuracy_report
```

```
from visualization import (save_all_figures, export_timeseries_csv,  
                           export_event_log_csv)
```

```
from scenarios import run_all, print_comparison, export_comparison_csv
```

```
OUTDIR = os.path.dirname(os.path.abspath(__file__))
```

```
def main():
```

```
    print("=" * 60)
```

```
    print(" SMART GRID CASCADING FAILURE SIMULATION")
```

```
    print("=" * 60)
```

```
    # --- 1. Baseline run -----
```

```
    sim = GridSimulation(seed=42)
```

```
    results = sim.run()
```

```
    print("\nBaseline metrics:")
```

```
    for k, v in summarise(results).items():
```

```
        print(f" {k:18s}: {v}")
```

```
    # --- 2. Figures -----
```

```
    print("\nSaving figures...")
```

```
    for p in save_all_figures(sim, outdir=OUTDIR):
```

```
        print(f" saved {os.path.basename(p)}")
```

```
    # --- 3. CSV exports -----
```

```
    ts_csv = export_timeseries_csv(sim, os.path.join(OUTDIR, "simulation_results.csv"))
```

```
    ev_csv = export_event_log_csv(sim, os.path.join(OUTDIR, "event_log.csv"))
```

```
    print(f" exported {os.path.basename(ts_csv)} and {os.path.basename(ev_csv)}")
```

```
    # --- 4. Scenario comparison -----
```

```
    print("\nScenario comparison:")
```

```
    rows = run_all(seed=42)
```

```
    print_comparison(rows)
```

```
    cmp_csv = export_comparison_csv(rows, os.path.join(OUTDIR, "scenario_comparison.csv"))
```

```
    print(f"\n exported {os.path.basename(cmp_csv)}")
```

```
    # --- 5. Accuracy study -----
```

```
    report = accuracy_study(n_runs=30)
```

```
    print_accuracy_report(report)
```

```
    print("\nDone.")
```

```
if __name__ == "__main__":
```

```
    main()
```